

I need to ask clarification data requirement to those who are responsible for front end
How do I write these requirements over a mail?

Yes. I would make the email more specific than simply asking “what data do you need?” The important clarification from the frontend team is **how each frontend requirement consumes data**: continuous/live stream, query/historical, or both, and what variables/filters/time ranges are required.

You can send something like this:

Subject: Clarification Required – HMI Frontend Data Requirements and Data Access Pattern

Hi Team,

As part of defining the WP5.3 Process Twin/HMI architecture and the interfaces between the HMI Backend and the RXU applications, we would like to clarify the **data requirements and data access pattern for the HMI frontend requirements**.

For each HMI functionality/view, could you please clarify whether the frontend requires **continuous/live data, query-based/historical data, or a combination of both?**

For example, for a live KPI/TCP visualization, the frontend may require continuously updated data during an active AFP run. In contrast, functions such as historical run analysis or a Pareto dashboard may require a query over a selected period of stored data.

It would therefore be helpful if you could provide the following information for each applicable frontend requirement:

1. **Data access type** – Is the required data:
 - Continuous/live stream,
 - Query-based/historical, or
 - Both live and historical?
2. **Required variables/data fields** – What exact information is required by the frontend?
For example:
 - Actual/programmed TCP and joint values
 - KPI values such as tension, pressure, temperature, speed, etc.
 - Process/anomaly information
 - Machine-health/fault information
 - Predictive-maintenance outputs
 - Any calculated/aggregated values required for visualization
3. **Required identifiers and filtering dimensions** – Which identifiers should be available for filtering or drill-down? For example:
 - Product ID

- Part ID
 - Run ID
 - Layer ID
 - Course ID
 - Strip ID
 - Tow ID
 - Time range
 - KPI/process parameter
 - Machine/component ID
4. **Time-window requirements** – For views based on historical/aggregated data, what time period should be selectable or available? This is particularly relevant for functions such as the **Maintenance Pareto Dashboard**. For example, should the user be able to select a particular run, day/week/month, or a custom start/end time?
 5. **Update/refresh requirements for live views** – For continuous monitoring requirements, what update rate or refresh behavior is expected on the frontend? For example, does the UI need every incoming sample, or is a reduced/aggregated update rate sufficient for visualization?
 6. **Initial data when opening a view** – If a user opens a live view after the AFP process has already been running for some time, should the frontend show only data received from that point onward, or should it first display the data accumulated since the beginning of the current run/course and then continue with live updates?

This clarification will help us define the appropriate interface between the HMI Backend RXU application and the other RXU applications, particularly whether each requirement should use a **query/service-based interface, continuous publish/subscribe data, or a combination of both**.

For reference, we are currently considering cases such as:

HMI functionality	Point to clarify
TCP / Joint Deviation – 3D CAD	Live only, accumulated current-run data, historical replay, or combination?
KPI Trends / KPI on 3D CAD	Required KPIs, update rate, historical period and filtering dimensions?
Process Anomaly Dashboard	Live anomaly events, historical anomaly/KPI data, or both?
Maintenance Pareto Dashboard	Required source variables, aggregation criteria and selectable time period?
Predictive Maintenance Dashboard	Live/periodic prediction results, historical trends, required component filters?
Historical Run / Replay	Required data types and supported run/time filters?

This information will allow us to finalize the backend interfaces, protobuf message definitions, RXU Pub/Sub versus Service/Client communication, and database query requirements.

Thanks,
Rasitha

This version asks the frontend team for the information you actually need for the architecture. In particular, I would **definitely keep point 6**. It resolves exactly the question we were discussing earlier: if the user opens TCP deviation halfway through a run, does the frontend need the trajectory from the beginning of the run plus live data, or only new data from that moment onward?

Also, instead of asking them directly whether something should be “Pub/Sub or Service/Client,” I would ask them for **live/query/both and the expected behavior**. The RXU architecture team can then derive whether Pub/Sub, Service/Client, database retrieval, or a combination is appropriate.

We need to know when for what user will be requesting query and continuous

Exactly. Then the clarification to the frontend team should focus less on **what technology to use** and more on **when the user needs query-based data versus continuously updated data for each frontend requirement**.

You need answers to questions like: when the user opens a view, does it need a one-time query? After that, does it need continuous updates? When the user changes a filter, should a new query happen? Does the view need historical data, current-run accumulated data, live data, or some combination?

I would write the email like this:

Subject: Clarification on HMI Data Requirements – Query-Based vs Continuous Data

Hi Team,

As part of defining the WP5.3 HMI architecture and the interfaces between the HMI Backend and the RXU applications, we need some clarification regarding the **data access behaviour expected for each frontend requirement**.

In particular, we would like to understand **when the frontend/user requires query-based data and when continuous/live data is required**.

For each applicable HMI functionality, could you please clarify the expected behaviour in the following scenarios?

- **When the user opens a view/page:** Should the frontend request the available data once, or should it start receiving continuous updates?
- **During an active AFP run:** Should the displayed information update continuously as new data becomes available?
- **When the user changes a filter** such as Product ID, Part ID, Run ID, Layer ID, Course ID, Strip ID, Tow ID, KPI type, component, or time range: should a new query be performed using the selected criteria?
- **When a user opens a view after the process has already been running:** Should the view show the data accumulated from the beginning of the current run/course and then continue with live updates, or only show data generated after the view is opened?
- **For a completed/historical run:** What information should be retrievable through user queries and which filtering criteria/time ranges should be supported?
- **For dashboards based on a period of time**, such as Maintenance Pareto, what should trigger the query—for example, selection of run, date range, machine/component, fault category, etc.?

For example, we would like to clarify the expected data behaviour for the following frontend functions:

Frontend function	Query-based requirement to clarify	Continuous requirement to clarify
TCP / Joint Deviation – 3D CAD	Does the user query by run/course/layer/etc. to retrieve an existing trajectory/deviation?	During an active run, should TCP/joint deviation continue updating on the 3D view?
KPI Trends / KPI on 3D CAD	Does selecting KPI, Tow ID, Course ID, Layer ID, etc. trigger a query?	Should the selected KPI continuously update on the CAD during execution?
Process Anomaly	Does the user query previous anomalies by run/time/KPI/tow/course/severity?	Should newly detected anomalies appear automatically while the process is running?

Machine Health / Pareto	Does the user select a time period/component/fault type and request an aggregated Pareto result?	Is any machine-health information expected to update continuously?
Predictive Maintenance	Does the user query historical predictions for a selected machine/component/time period?	Should the latest prediction/health indicator continuously or periodically update?
Historical Run / Replay	Which IDs/time ranges should the user use to retrieve a previous run?	Is continuous data applicable during replay, or is this purely query-based?

We would also like to understand the **required variables/data fields for each view** and the expected filtering dimensions, particularly:

Product ID, Part ID, Run ID, Program ID, Layer ID, Course ID, Strip ID, Tow ID, KPI type, Machine/Component ID and Time Range.

The purpose of this clarification is to identify, for each frontend requirement:

User action → required data → query or continuous update → required filters → expected visualization behaviour.

This will allow us to define the appropriate RXU interfaces and subsequently decide where **Service/Client (query-based)** and **Publish/Subscribe (continuous data)** communication are required.

Thanks,
Rasitha

The last sentence is important: **don't ask the frontend team to decide "Service/Client vs Pub/Sub."** Ask them to tell you the **user behaviour and data behaviour**. From that, you can derive the RXU communication architecture.

For example, if they say, *"When the operator selects Course 7 and Tow 12, show all tension data already generated for that course and keep updating until the course completes,"* then you can architect it as **initial query + continuous stream**. If they say, *"User selects last month and component X to generate Pareto,"* that is naturally **query-based**.

May be you can first mail you have and the points which I asked just before together?

Yes. I would combine them into **one concise clarification email** covering both **data requirements** and **when the frontend expects query-based versus continuous data**.

Subject: Clarification on HMI Frontend Data Requirements and Data Access Behaviour

Hi Team,

As part of defining the WP5.3 HMI architecture and the interfaces between the HMI Backend and the RXU applications, we would like to clarify the **data requirements and expected data access behaviour for the frontend requirements**.

In particular, we need to understand **what data/variables are required for each HMI functionality, and when the frontend expects query-based data versus continuous/live data**.

Could you please provide clarification on the following aspects for each applicable frontend requirement?

1. Required data / variables

What data is required to support the particular visualization or dashboard? For example, TCP/joint information, process KPIs, anomaly information, machine-health information, maintenance/prediction information, etc.

2. Query-based vs continuous data requirement

For each functionality, we would like to understand when data is expected to be:

- **Query-based** – requested based on a user selection or specific criteria;
- **Continuous/live** – continuously updated while the process is running;
- **Both** – initial/accumulated data followed by continuous updates.

3. Behaviour when the user opens a view

When a user opens a particular page/view, should the frontend:

- request the available data once,
- start receiving continuous data,
- or retrieve the existing/current-run data first and then continue receiving live updates?

4. For example, if an AFP run has already been running for 20 minutes when the user opens the TCP deviation view, should the frontend display the deviation accumulated from the beginning of the run and then continue with live updates, or only display new data from the point at which the view is opened?

5. User-triggered queries and filters

We would also like to identify **which user actions are expected to trigger a new data query**. For example, selection/change of:

- Product ID
- Part ID
- Run ID
- Layer ID
- Course ID
- Strip ID
- Tow ID
- KPI/process parameter
- Machine/component
- Time range
- Severity/event type, where applicable

6. **Historical/time-period requirements**

For historical views or calculations over a period of time, what time-selection options are expected?

This is particularly relevant for functionality such as the **Maintenance Pareto Dashboard**. For example, should the user be able to request Pareto information for a selected run, day/week/month, custom time range, machine/component, fault category, etc.?

7. **Continuous update/refresh requirements**

For views requiring live monitoring, what update behaviour is expected? Do all incoming samples need to be visualized, or is a particular UI refresh/aggregation rate expected?

To make the expected behaviour clearer, it would be helpful to clarify it for the main functionalities as below:

HMI Functionality	Query-based behaviour	Continuous/live behaviour
TCP / Joint Deviation – 3D CAD	What should the user be able to query/filter: run, part, layer, course, etc.? Should previous/current-run deviation be retrieved when opening the view?	Should the 3D trajectory/deviation update continuously during an active run?
KPI Trends / KPI on 3D CAD	Does selecting KPI, Tow ID, Course ID, Layer ID, Strip ID, etc. trigger retrieval of the corresponding data?	Should the selected KPI continue updating spatially during execution?

Process Anomaly / Time-Series View	Should users query previous anomalies using run, KPI, tow, course, severity, time range, etc.?	Should newly detected KPI variations/anomalies automatically appear during an active run?
Machine Health / Maintenance Pareto	Which time period, component, fault category, severity, etc. should the user be able to select for Pareto calculation?	Which machine-health information, if any, needs continuous display?
Predictive Maintenance Dashboard	Should historical prediction/health information be queryable by component/time period?	Should the latest prediction/health indicators update continuously or periodically?
Historical Run / Replay	What IDs and time ranges should be supported for retrieving a previous run?	Is any continuous/replay update expected after the historical data is loaded?

The objective is to establish, for each frontend requirement:

Frontend functionality → required variables/data → user/query trigger → query-based or continuous data → required filters → expected visualization/update behaviour.

This clarification will help us define the interfaces between the HMI Backend and the RXU applications and subsequently determine where **query-based Service/Client communication, continuous Publish/Subscribe communication, or a combination of both** is required.

Thanks,
Rasitha